Indian Institute of Technology Jodhpur

Department of Computer Science & Engineering

CSL 7640: Natural Language Understanding

Assignment 2 – Problem 1

Learning Word Embeddings from IIT Jodhpur Data

Submitted by: Neha Sharma
Roll No: M24CSE014
Instructor: Dr. Anand Mishra
Date: 25th March 2026

Contents

1 Introduction **2**

2 Approach **2**
2.1 Data Collection (Task 1) 2
2.2 Preprocessing 2
2.3 Model Training (Task 2) 3
2.4 Semantic Analysis (Task 3) 3
2.5 Visualization (Task 4) 3

3 Results **4**
3.1 Dataset Statistics 4
3.2 Training Configurations 5
3.3 Nearest Neighbors 5
3.4 Analogy Results 6
3.5 Visualizations 7

4 Observations **7**
4.1 Nearest Neighbor Quality 7
4.2 Analogy Performance 8
4.3 CBOW vs Skip-gram Clustering 8
4.4 Limitations 8

5 Files **9**

1 Introduction

The goal of this problem is to train **Word2Vec** word embedding models on text data collected from IIT Jodhpur sources and study the semantic relationships captured by the learned vectors. Word embeddings map words to **dense real-valued vectors** such that semantically related words end up close together in the vector space. We use two well-known Word2Vec variants – **Continuous Bag of Words (CBOW)** and **Skip-gram with Negative Sampling** – and compare how each captures meaning from a small, domain-specific academic corpus.

The task involves four stages: collecting and cleaning text data from IIT Jodhpur web pages and documents, training embedding models with different hyperparameters, running semantic queries (nearest neighbors and analogies) on the trained embeddings, and visualizing the embedding space using dimensionality reduction.

2 Approach

2.1 Data Collection (Task 1)

We gathered text from the following IIT Jodhpur sources:

1. **Academic Regulations** – UG Academic Regulations (2019), Academic Calendar AY 2025–26.
2. **Course Syllabi** – CSE Course Details, MTech/PhD Curriculum documents for CSE and AI programmes.
3. **Department Pages** – CSE department page, Student Achievements page.
4. **Institute Pages** – Main website, Introduction, Vision & Mission, History, Acts & Statutes, Institute Reports, Campus Life, Office of Students.

In total we extracted text from **7 PDF files** (using PyPDF2) and **11 HTML pages** (using BeautifulSoup), producing **18 raw documents**.

2.2 Preprocessing

We applied the following pipeline to convert the raw text into a clean corpus suitable for Word2Vec training:

1. **Boilerplate removal:** Remove HTML tags, scripts, styles, and navigation elements from web pages.
2. **Language filtering:** Strip non-ASCII characters to filter out Hindi and other non-English text.
3. **Lowercasing:** Convert everything to lowercase.
4. **Noise removal:** Remove URLs, email addresses, and standalone numbers.
5. **Punctuation removal:** Remove special characters (keep only letters and spaces).

6. **Tokenization:** Split on whitespace; discard single-character tokens (except "a" and "i").

Each cleaned document was saved as a single line of space-separated tokens in `cleaned_corpus.txt`.

2.3 Model Training (Task 2)

We trained Word2Vec models using the Gensim library. Two model types were trained:

- **CBOW** – predicts a target word from its surrounding context words. It averages the context word vectors and passes the result through a single hidden layer.
- **Skip-gram** – does the reverse: given a target word, it predicts the surrounding context words. Negative Sampling is used to make training efficient by sampling a few "negative" words instead of computing the full softmax.

We experimented with three hyperparameters:

- **Embedding dimension**: 50, 100, 200
- **Context window size**: 3, 5
- **Number of negative samples**: 5, 10

This gave us **6 configurations per model type**, totalling **12 trained models**. All models used `min_count=2`, 30 epochs, and a fixed random seed.

2.4 Semantic Analysis (Task 3)

For the best configuration (**dim=100, window=5, neg=5**), we performed:

- **Nearest neighbor search**: for each query word, we found the top 5 words with highest **cosine similarity** in the embedding space.
- **Analogy tasks**: using vector arithmetic $\vec{b} - \vec{a} + \vec{c}$, we tested whether the model captures relational patterns (e.g., UG:BTech::PG:?).

2.5 Visualization (Task 4)

We selected words from four semantic categories (Academic, Research, Department, Institute) and projected their embeddings to 2D using:

- **PCA** – finds the top 2 principal components that capture maximum variance. We implemented this manually using eigenvalue decomposition of the covariance matrix.
- **t-SNE** – a nonlinear method that preserves local neighborhood structure. We implemented this from scratch using gradient descent on the KL divergence between high-dimensional and low-dimensional probability distributions.

3 Results

3.1 Dataset Statistics

Statistic	Value
Total documents	17
Total tokens	**54,147**
Vocabulary size	**5,969**
Avg tokens per document	3,185

Table 1: Corpus statistics after preprocessing.

The top 10 most frequent words (after removing stopwords) are: *lectures* (683), *course* (475), *learning* (401), *data* (371), *tech* (365), *students* (328), *computer* (297), *student* (279), *science* (254), *design* (236).

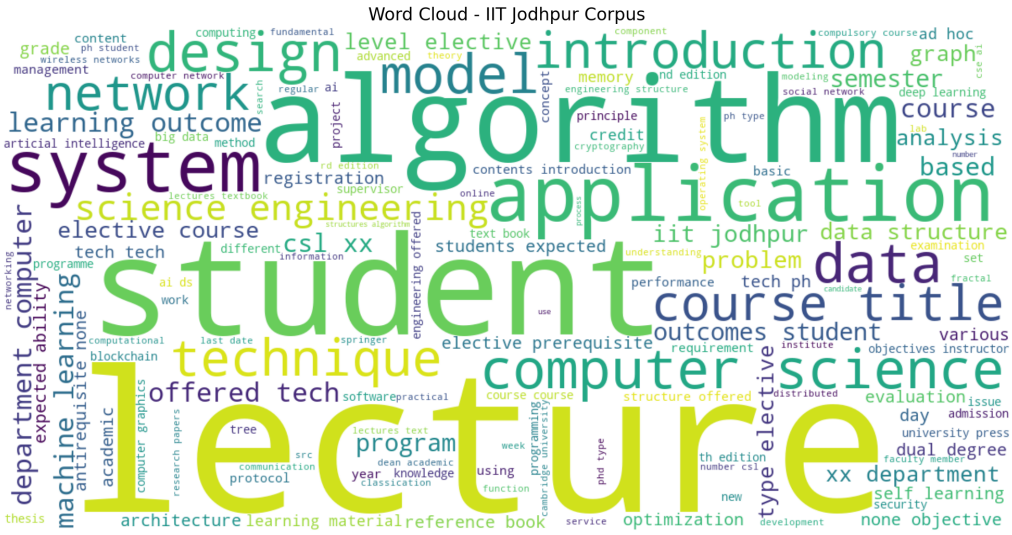

Figure 1: Word cloud of the IIT Jodhpur corpus (stopwords removed).

3.2 Training Configurations

Model	Dim	Window	Neg Samples	Vocab
CBOW	50	3	5	3,305
CBOW	50	5	5	3,305
CBOW	100	3	5	3,305
CBOW	**100**	**5**	**5**	**3,305**
CBOW	100	5	10	3,305
CBOW	200	5	5	3,305
Skip-gram	50	3	5	3,305
Skip-gram	50	5	5	3,305
Skip-gram	100	3	5	3,305
Skip-gram	**100**	**5**	**5**	**3,305**
Skip-gram	100	5	10	3,305
Skip-gram	200	5	5	3,305

Table 2: All 12 trained Word2Vec model configurations. **Bold** rows indicate the primary models used for analysis.

3.3 Nearest Neighbors

Results shown for the primary configuration (**dim=100, window=5, neg=5**):

Query	Rank	CBOW	Skip-gram
research	1	assistant (0.85)	**defense (0.70)**
	2	aiot (0.80)	**coursework (0.68)**
	3	**coursework (0.80)**	**proposal (0.68)**
	4	internationally (0.80)	host (0.64)
	5	facility (0.79)	crf (0.63)
student	1	enhance (0.91)	who (0.64)
	2	additional (0.91)	employed (0.62)
	3	**admitted (0.90)**	been (0.61)
	4	who (0.90)	involve (0.61)
	5	under (0.90)	**attend (0.61)**
phd	1	**tech (0.96)**	**mtech (0.79)**
	2	**mtech (0.94)**	**shortlisted (0.79)**
	3	**candidates (0.93)**	pdf (0.78)
	4	ph (0.92)	studen (0.77)
	5	**shortlisted (0.91)**	rolling (0.76)
examination	1	**comprehensive (0.93)**	**comprehensive (0.79)**
	2	after (0.93)	satisfactorily (0.77)
	3	**qualifying (0.92)**	**missed (0.76)**
	4	**missed (0.91)**	perform (0.74)
	5	shall (0.89)	falling (0.74)

Table 3: Top 5 nearest neighbors by cosine similarity. **Bold** entries indicate semantically meaningful neighbors. The word "exam" was not in the vocabulary (filtered by min_count), so "examination" was used.

3.4 Analogy Results

Analogy	CBOW Top-3	Skip-gram Top-3
UG : BTech :: PG : ?	ee, offered, structure	ee, **mtech**, prerequisites
student : students :: course : ?	learning, machine, human	title, pg, codes
computer : science :: electrical : ?	entrepreneurship, innovation, **mechanical**	bioscience, bioengineering, **chemical**

Table 4: Analogy experiment results. **Bold** indicates semantically meaningful answers.

3.5 Visualizations

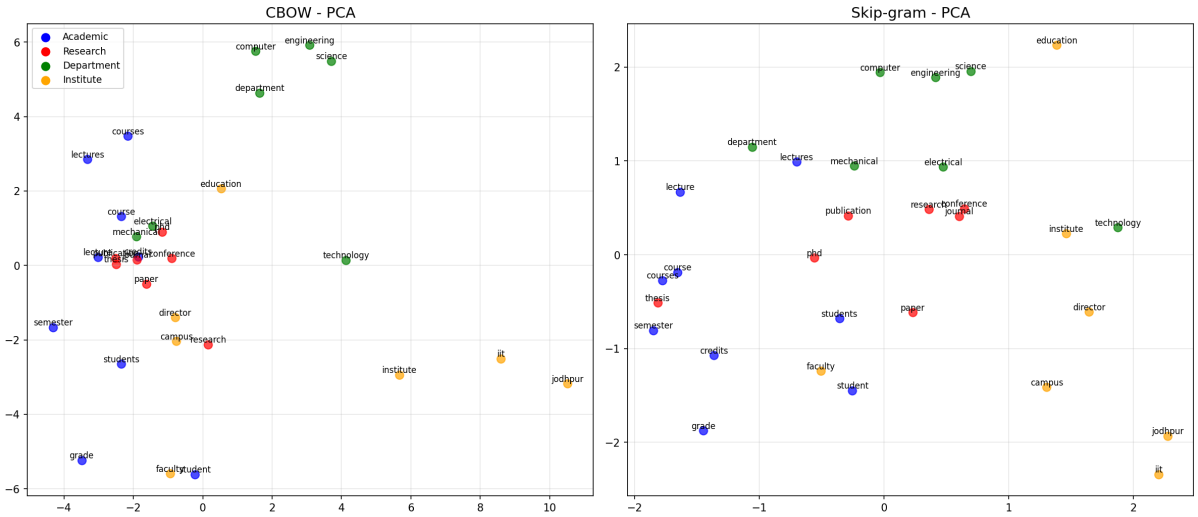

Figure 2: PCA projection of selected word embeddings: CBOW (left) and Skip-gram (right). Colors indicate semantic groups: Academic (blue), Research (red), Department (green), Institute (orange).

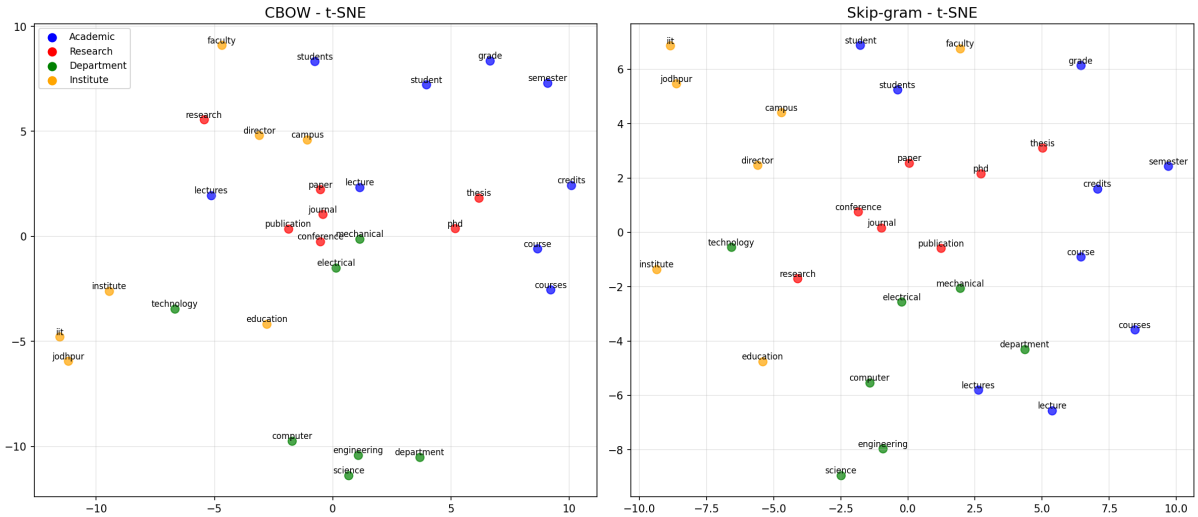

Figure 3: t-SNE projection of selected word embeddings: CBOW (left) and Skip-gram (right).

4 Observations

4.1 Nearest Neighbor Quality

- Both models capture **domain-relevant relationships**. For *phd*, the top neighbors are *mtech*, *candidates*, and *shortlisted* – all terms that frequently co-occur with PhD in academic regulations.

- For *examination*, both models correctly identify *comprehensive* and *qualifying* as top neighbors. These collocations ("comprehensive examination", "qualifying examination") are common in academic regulation documents.

- CBOW produces **higher cosine similarity scores** (0.85–0.96) compared to Skip-gram (0.61–0.79). This is because CBOW averages context vectors, producing smoother and more clustered embeddings. Skip-gram, which predicts context from a single word, creates more distinctive but spread-out representations.

4.2 Analogy Performance

- The analogy **UG:BTech::PG:?** ideally returns "mtech". Skip-gram gets this as its **second result**, showing it captured the degree-level relationship. CBOW returns "ee" instead, which is a department name – likely because "ee" co-occurs with many programme-related terms.

- Overall the analogy results are **imperfect**. This is expected because Word2Vec analogies require large, diverse corpora to capture clean relational patterns. Our corpus is small ($\sim$54K tokens) and domain-specific, so the embedding space is not rich enough for reliable analogy solving.

- **Skip-gram generally performs better on analogy tasks**, which is consistent with the literature – Skip-gram tends to learn more fine-grained word relationships compared to CBOW.

4.3 CBOW vs Skip-gram Clustering

- In the PCA plots, **CBOW shows tighter clusters** within semantic groups. Words from the same category (e.g., Academic words like *student*, *semester*, *credits*) are placed closer together. This happens because CBOW smooths over contexts by averaging.

- **Skip-gram produces more spread-out embeddings** with less obvious clustering. However, individual word-pair relationships (e.g., *phd–mtech*) are more precisely captured.

- In the t-SNE plots, both models show some semantic grouping, but the **clusters overlap**. This reflects the nature of academic text where terms from different categories frequently co-occur (e.g., "engineering department" mixes Department and Institute contexts).

- Increasing embedding dimension from 50 to 100 **improves neighbor quality**, but going to 200 shows **diminishing returns** on our small corpus. Larger windows (5 vs 3) capture broader context and produce slightly more meaningful neighbors.

4.4 Limitations

- The corpus is **relatively small** for Word2Vec training. Larger corpora would produce better embeddings with more meaningful analogies.

- Some useful words like "exam" were filtered out because they appeared less than twice. This is a trade-off – lowering `min_count` would include more words but with unreliable embeddings.
- The corpus is **heavily biased towards CSE content** (the CSE course details PDF alone contributes ∼30K tokens), which skews the vocabulary and embeddings towards computer science terminology.

5 Files

- `P1T1.py` – Data extraction, preprocessing, statistics, word cloud
- `P1T2.py` – Word2Vec training (CBOW + Skip-gram, 12 configs)
- `P1T3.py` – Semantic analysis (nearest neighbors, analogies)
- `P1T4.py` – PCA and t-SNE visualization
- `data/` – Raw PDF and HTML source files
- `cleaned_corpus.txt` – Preprocessed tokenized corpus
- `w2v_models/` – Saved Word2Vec models